Revive

Dossier de lancement — M1 MIAGE Apprentissage — v0.1.0-SNAPSHOT

Jorys KANCEL, SALEKHA FALL, SARAH METREF, CAMILIA SABA, MOHAMED GUEYE · 20 octobre 2026

01 Équipe, comptes développeurs et dépôt

MEMBRE	COMPTE GITHUB	EMAIL DES COMMITS (`GIT CONFIG USER.EMAIL`)
Jorys KANCEL	`@Sinie-Kun`	jorys.kancel@gmail.com
SALEKHA FALL	`@salekhaf`	zulekhfall@gmail.com
SARAH METREF	`@SarahMETREF`	metrefsarah@gmail.com
CAMILIA SABA	`@fromwonderland`	camilia200@outlook.fr
MOHAMED GUEYE	`@Moh-svg04`	gueyemoe04@gmail.com

Dépôt : https://github.com/Moh-svg04/reUseAi-projet-m1-miage · **page de projet** : https://Moh-svg04.github.io/reUseAi-projet-m1-miage/.

02 Projet et caractère innovant

Revive est une application d'aide à la décision destinée aux associations et aux équipes de collecte qui organisent le réemploi d'objets. Elle vise à coordonner des campagnes de collecte à domicile, à prendre en compte les capacités des véhicules et à aider à répartir les objets récupérés entre les associations selon leurs besoins.

Le problème traité ne consiste pas seulement à enregistrer des objets : il faut choisir quels lots collecter, regrouper les adresses, construire des chargements compatibles avec les capacités en poids et en volume, planifier les tournées et réagir aux imprévus. Une organisation manuelle devient difficile lorsque plusieurs contraintes doivent être respectées simultanément.

L'entité pivot envisagée est le **lot d'objets**, rattaché à une campagne et caractérisé notamment par sa catégorie, son état, son poids, son volume, sa localisation et son statut. Les campagnes, véhicules, équipes, besoins des associations, tournées et affectations compléteraient le modèle métier.

Le caractère innovant de Revive reposerait sur l'intégration d'algorithmes d'aide à la décision dans un parcours applicatif complet : priorisation multicritère, regroupement géographique, chargement sous contraintes, optimisation des tournées et affectation des objets.

L'équipe envisage également d'étudier une composante de **Machine Learning** afin d'estimer les besoins futurs des associations à partir de l'historique des demandes, des collectes et des redistributions. Un modèle de régression supervisée pourrait être envisagé pour prédire les quantités demandées par catégorie d'objets. Sa pertinence dépendrait

toutefois de la disponibilité et de la qualité des données. En l'absence de données historiques suffisantes, des données synthétiques pourraient servir aux premiers essais, sans permettre à elles seules de conclure aux performances du modèle en conditions réelles.

Ces propositions constituent des pistes de conception qui pourront être précisées ou adaptées au cours du projet. Les algorithmes retenus devront notamment tenir compte des contraintes métier, de la faisabilité des solutions proposées et de la possibilité pour les responsables de comprendre et de modifier les résultats.

Le géocodage pourrait initialement être renseigné manuellement. Le calcul des distances routières s'appuierait sur un service de routage compatible avec les données OpenStreetMap. Les appels externes seraient isolés derrière un adaptateur et les tests pourraient utiliser des données déterministes afin de rester reproductibles.

03 Scénarios d'usage

- **En tant que responsable de campagne**, je veux pouvoir classer les lots à collecter selon des critères pondérés, afin de décider quelles collectes traiter en priorité.
- **En tant que responsable logistique**, je veux pouvoir regrouper les adresses proches et constituer des chargements respectant le poids et le volume autorisés, afin de préparer des collectes réalisables.
- **En tant que membre d'une équipe de collecte**, je veux pouvoir consulter une tournée ordonnée selon les distances routières, afin de limiter les déplacements inutiles.
- **En tant que responsable d'association**, je veux pouvoir obtenir une proposition de répartition des objets en fonction des besoins et des stocks disponibles, afin de mieux répondre aux demandes des bénéficiaires.
- **En tant que responsable d'association**, je veux pouvoir consulter une estimation des besoins futurs par catégorie d'objets, si les données disponibles le permettent, afin d'anticiper les prochaines campagnes et d'adapter les priorités de redistribution.
- **En tant que responsable logistique**, je veux pouvoir recalculer un plan lorsqu'un véhicule ou un point de collecte devient indisponible, afin de limiter les conséquences de l'imprévu.
- **En tant qu'administrateur**, je veux pouvoir consulter les contraintes violées et les indicateurs d'un plan, afin d'examiner sa faisabilité avant de le valider.

04 Plan des features

Une feature a un effet sur **la BD, le back, l'API REST et le front**. Chaque feature est portée par **une seule personne** et repose sur **un algorithme identifié**. La répartition ci-dessous est une proposition initiale tournante : elle devra être validée par l'équipe et chaque fiche devra préciser les contraintes, les cas limites et la complexité de l'algorithme réellement implémenté.

VERSION	MEMBRE 1	MEMBRE 2	MEMBRE 3	MEMBRE 4
v0.1.0 24/11	Feature — algorithme			
v0.2.0 08/12				

VERSION	MEMBRE 1	MEMBRE 2	MEMBRE 3	MEMBRE 4
v1.0.0 19/01				

Les complexités seront documentées dans les fiches de features en fonction des structures de données et de l'implémentation choisies. Les fonctionnalités dépendant de données historiques seront évaluées avec prudence : les jeux synthétiques permettent de tester l'algorithme, mais ne suffisent pas à revendiquer une précision prédictive réelle. La distinction entre fonctionnalités devra être justifiée par des objectifs et contraintes différents, notamment pour l'affectation initiale et la répartition globale, ainsi que pour la comparaison et la validation des plans.

05 Architecture

```
Front  --HTTP/JSON-->  API REST  -->  Back (cas d'utilisation)  --JPA-->  BD
(web)                (Spring MVC)      (Java 21, hexagonal)           (PostgreSQL)
```

COUCHE	CHOIX	REMARQUES
Front	application web : HTML, CSS, JavaScript natif	client de l'API REST uniquement ; dossier `front/`
API REST	Spring Boot 4 (Spring MVC), OpenAPI / Swagger UI	versionnée sous `/api/v1`, erreurs au format RFC 9457
Back	Java 21, Spring Boot 4, Spring Data JPA, Bean Validation	**architecture hexagonale**
BD	PostgreSQL 17 (H2 pour les tests)	schéma versionné par **Flyway**, une migration par feature
Qualité	JUnit 5, AssertJ, ArchUnit, JaCoCo, Codecov, SonarCloud	intégration continue GitHub Actions
Conteneurs	Docker Compose	BD dès v0.1.0, application en v0.2.0

Le modèle métier commun s'organise autour du lot d'objets, lié à une campagne, une localisation, un statut de collecte et, après récupération, à une affectation éventuelle. Les véhicules portent des capacités en poids et en volume ; les associations expriment des besoins par catégorie et quantité. Les migrations Flyway versionnent les évolutions du schéma.

Qualité du code. Au-delà de l'architecture hexagonale :

- **Clean Code** : noms métier explicites, fonctions courtes, pas de duplication, commentaires qui expliquent le pourquoi ;
- **SOLID**, objets valeur immuables et invariants vérifiés à la construction ;
- **design patterns** quand ils servent, nommés dans les fiches de features ;
- conventions **REST** et règles de sécurité **OWASP** ;
- quality gate SonarCloud au vert.

06 Démarche pour chaque feature

Les étapes s'enchaînent **dans cet ordre, sans en sauter** : on ne commence une étape que lorsque la précédente est terminée.

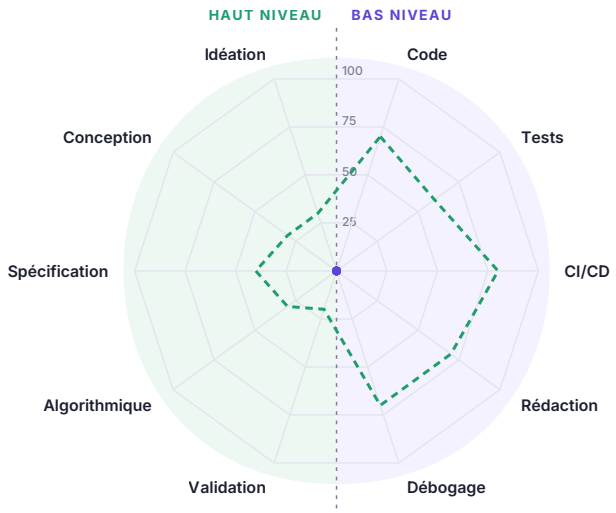
ÉTAPE	CE QU'ON FAIT	TERMINÉ QUAND
1. Idéation	Besoin et scénario ; **algorithme retenu**, complexité, alternatives ; impact sur le schéma et l'API ; critères d'acceptation. Issue + fiche `doc/features/` .	La fiche est validée par un autre membre.
2. Back	Migration Flyway ; **domaine** en Java pur (modèle, règles, algorithme) ; **application** (ports, service) ; adaptateur de persistance.	Le code compile et ArchUnit passe.
3. Tests	Unitaires de l'algorithme (cas limites, invariants, propriétés) ; services avec faux adaptateurs ; intégration de la persistance.	`./gradlew build` passe ; chaque critère d'acceptation est couvert.
4. API REST	Endpoints `/api/v1` , DTO validés, erreurs `problem+json` , OpenAPI ; tests de l'API.	Visible dans Swagger UI ; tests de l'API verts.
5. Front	Écrans qui consomment **uniquement** l'API.	Scénario jouable de bout en bout ; PR relue ; fiche complétée avec son araignée.

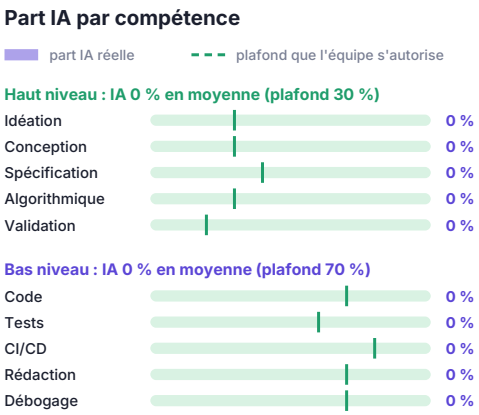
07 Protocole d'usage de l'IA

Principe : l'IA assiste, l'humain reste l'auteur. On ne laisse jamais l'IA tout faire sans comprendre : tout ce qui est comité a été relu et compris par son auteur, capable de l'expliquer, de le modifier et de le refaire sans aide. Les consignes des assistants sont ce dans `CLAUDE.md` .

ACTIVITÉ	CE QUE FAIT L'IA	CE QUE FAIT L'HUMAIN
Idéation et conception	Critique les idées, propose des options	Choisit, justifie, rédige la fiche
Algorithme	Explique, relit, signale les cas oubliés	**Conçoit et écrit le cœur de l'algorithme**
Code technique	Peut générer un premier jet	Relit chaque ligne, adapte, assume
Tests	Suggère des cas	Définit cas limites et propriétés ; vérifie qu'un test échoue quand on casse le code
CI/CD, conteneurisation	Peut générer workflows et Dockerfile	Comprend chaque étape, vérifie l'exécution, gère les secrets
Documentation	Aide à rédiger et mettre en forme	Valide le contenu, vérifie les faits

08 Annexe : part IA / part humaine





Valeurs dans `doc/part-ia.json`, régénérées à chaque compilation.